Problem 1:

Example 1=

```
  #include <stdio.h>
  int find_median(int *arr, int size) {
  int mid = (size - 1) / 2;

  for (int i = 0; i <= mid; i++) {
     int min_index = i;

     for (int j = i + 1; j < size; j++) {
        if (arr[j] < arr[min_index]) {
           min_index = j;
        }
     }
     int temp = arr[i];
     arr[i] = arr[min_index];
     arr[min_index] = temp;
  }
  return arr[mid];
}
 int main() {
  int arr[] = {7, 2, 10, 9, 1};
  int size = 5;
  int median = find_median(arr, size);
  printf("Median = %d\n", median);

  return 0;
}
```

Output=

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
    int find_median(int *arr, int size) {
    int mid = (size - 1) / 2;

    for (int i = 0; i <= mid; i++) {
        int min_index = i;

        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_index]) {
                min_index = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min_index];
        arr[min_index] = temp;
    }
    return arr[mid];
}
  int main() {
    int arr[] = {7, 2, 10, 9, 1};
    int size = 5;
    int median = find_median(arr, size);
    printf("Median = %d\n", median);
```

input

```
Median = 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Example 2 =

```
#include <stdio.h>
   int find_median(int *arr, int size) {
   int mid = (size - 1) / 2;
      for (int i = 0; i <= mid; i++) {
       int min_index = i;
           for (int j = i + 1; j < size; j++) {
           if (arr[j] < arr[min_index]) {
               min_index = j;
           }
       }
       int temp = arr[i];
       arr[i] = arr[min_index];
       arr[min_index] = temp;
   }
return arr[mid];
}
int main() {
   int arr[] = {4, 1, 8, 3};
   int size = 4;
   int median = find_median(arr, size);
   printf("Median = %d\n", median);
 return 0;
}
```

Output:

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>

int find_median(int *arr, int size) {
    int mid = (size - 1) / 2;

    for (int i = 0; i <= mid; i++) {
        int min_index = i;

        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[min_index]) {
                min_index = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[min_index];
        arr[min_index] = temp;
    }

    return arr[mid];
}

int main() {
    int arr[] = {4, 1, 8, 3};
```

input

```
Median = 3


...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 2

Example 1=

```
#include <stdio.h>
   int find_kth_largest(int *arr, int size, int k) {
      for (int i = 0; i < k; i++) {
        for (int j = 0; j < size - i - 1; j++) {
          if (arr[j] > arr[j + 1]) {
              int temp = arr[j];
              arr[j] = arr[j + 1];
              arr[j + 1] = temp;
          }
      }
   }
   return arr[size - k];
}
  int main() {
   int arr[] = {3, 1, 7, 4, 9, 2};
   int size = 6;
   int k = 2;
   int result = find_kth_largest(arr, size, k);
   printf("K-th largest element = %d\n", result);
   return 0;
}
```

Output=

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>

int find_kth_largest(int *arr, int size, int k) {
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    return arr[size - k];
}

int main() {
    int arr[] = {3, 1, 7, 4, 9, 2};
    int size = 6;
    int k = 2;

    int result = find_kth_largest(arr, size, k);

    printf("K-th largest element = %d\n", result);
```

input

```
K-th largest element = 7


...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 3:

```
#include <stdio.h>
  void insert_sorted(int *out, int *size, int value) {
    int i = *size - 1;
    while (i >= 0 && out[i] > value) {
        out[i + 1] = out[i];
        i--;
    }
    out[i + 1] = value;
    (*size)++;
}
void merge_sorted(int *a, int n, int *b, int m, int *out) {
    int size = 0;
     for (int i = 0; i < n; i++) {
        insert_sorted(out, &size, a[i]);
    }
      for (int i = 0; i < m; i++) {
        insert_sorted(out, &size, b[i]);
    }
}
int main() {
    int a[] = {5, 1, 9};
    int b[] = {3, 7, 2, 6};
```

```
    int n = 3, m = 4;
    int out[100];
    merge_sorted(a, n, b, m, out);
    printf("Sorted output: ");
    for (int i = 0; i < n + m; i++) {
        printf("%d ", out[i]);
    }
 return 0;
}
```

Output=

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>

void insert_sorted(int *out, int *size, int value) {
    int i = *size - 1;

    while (i >= 0 && out[i] > value) {
        out[i + 1] = out[i];
        i--;
    }

    out[i + 1] = value;
    (*size)++;
}

void merge_sorted(int *a, int n, int *b, int m, int *out) {
    int size = 0;

    for (int i = 0; i < n; i++) {
        insert_sorted(out, &size, a[i]);
    }

    for (int i = 0; i < m; i++) {
        insert_sorted(out, &size, b[i]);
```

input

```
Sorted output: 1 2 3 5 6 7 9

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 4.1

```
#include <stdio.h>
  long long factorial(int n) {
    long long result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
      }
   return result;
}
int main() {
    int n;
    scanf("%d", &n);
    printf("%lld\n", factorial(n));
    return 0;
}
```

Problem 4.2

```
#include <stdio.h>
   long long fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;

    return fibonacci(n - 1) + fibonacci(n - 2);
}
int main() {
```

```
    int n;
    scanf("%d", &n);
    printf("%lld\n", fibonacci(n));
    return 0;
}
```

Problem 4.3

```
#include <stdio.h>
  int count_digits(int n) {
    if (n == 0) {
        return 1;
    }
    if (n < 10) {
        return 1;
    }
 return 1 + count_digits(n / 10);
}
int main() {
    printf("%d\n", count_digits(0));
    printf("%d\n", count_digits(9));
    printf("%d\n", count_digits(100));
    printf("%d\n", count_digits(98765));
    return 0;
}
```

Problem 4.4

```
#include <stdio.h>
  double power(double base, int exp) {
    double result = 1.0;
    for (int i = 0; i < exp; i++) {
        result *= base;
    }
   return result;
}
int main() {
    printf("%.3f\n", power(2.0, 0));
    printf("%.3f\n", power(3.0, 4));
    printf("%.3f\n", power(2.5, 3));
    return 0;
}
```

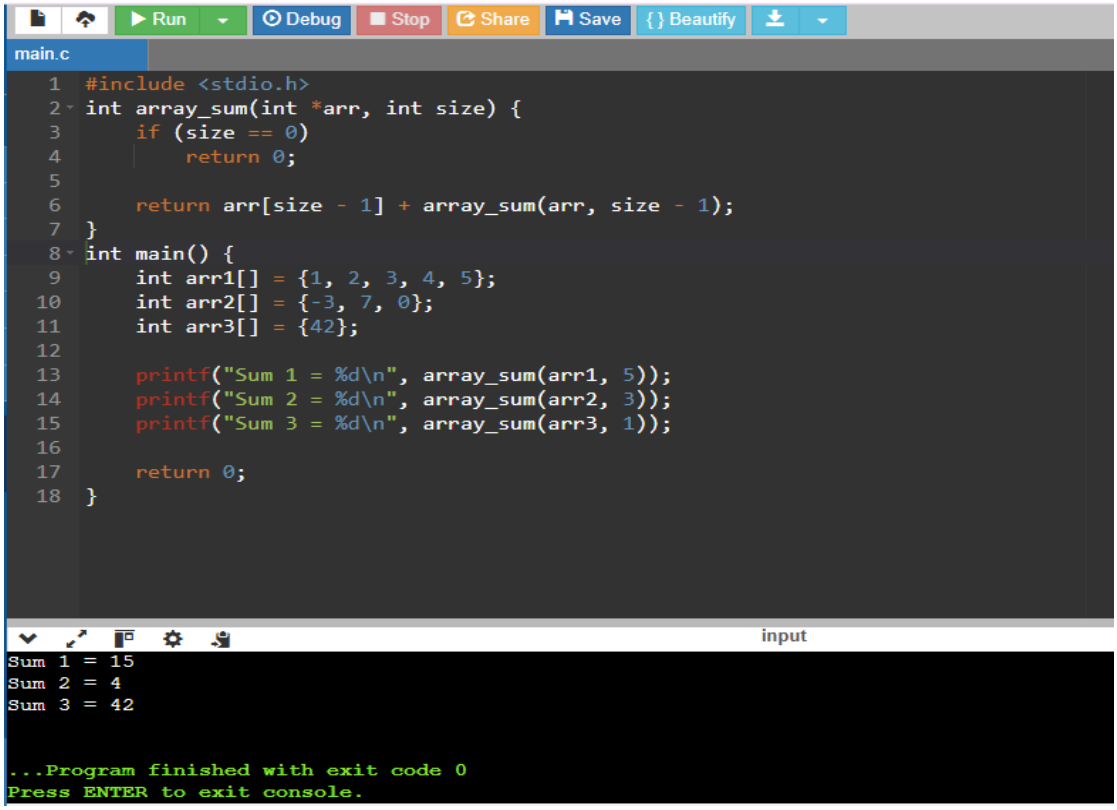
Problem 4.5

```
int is_palindrome(char *str, int left, int right) {
    if (left >= right) return 1;
    if (str[left] != str[right]) return 0;
    return is_palindrome(str, left + 1, right - 1);
}
```

Problem 4.6

```
#include <stdio.h>
  int array_sum(int *arr, int size) {
    if (size == 0)
      return 0;
      return arr[size - 1] + array_sum(arr, size - 1);
}
int main() {
   int arr1[] = {1, 2, 3, 4, 5};
   int arr2[] = {-3, 7, 0};
   int arr3[] = {42};
   printf("Sum 1 = %d\n", array_sum(arr1, 5));
   printf("Sum 2 = %d\n", array_sum(arr2, 3));
   printf("Sum 3 = %d\n", array_sum(arr3, 1));
   return 0;
}
```

Output=

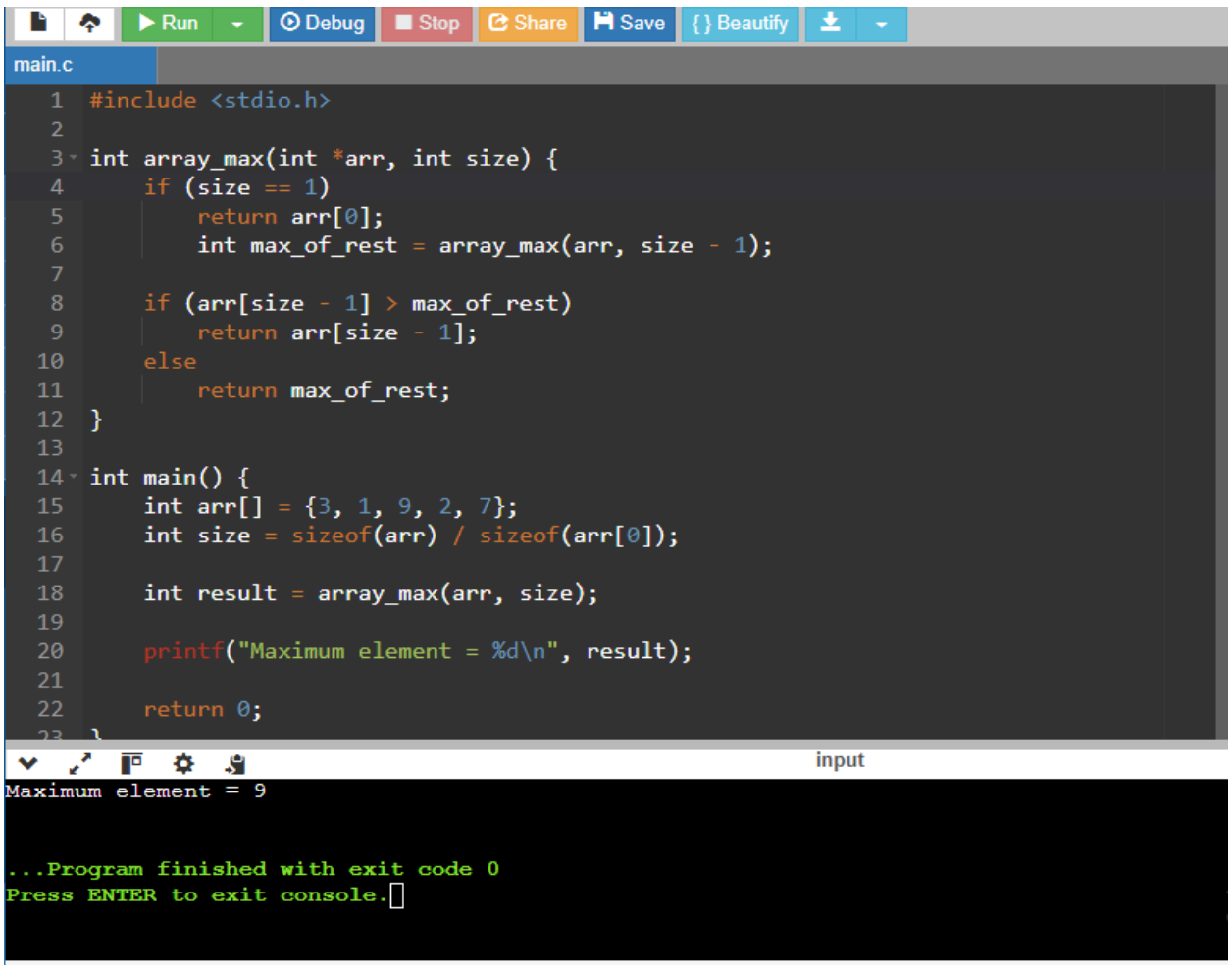


Problem 4.7

```
#include <stdio.h>
 int array_max(int *arr, int size) {
   if (size == 1)
      return arr[0];
      int max_of_rest = array_max(arr, size - 1);
      if (arr[size - 1] > max_of_rest)
      return arr[size - 1];
   else
      return max_of_rest;
}
int main() {
```

```
    int arr[] = {3, 1, 9, 2, 7};
    int size = sizeof(arr) / sizeof(arr[0]);
    int result = array_max(arr, size);
    printf("Maximum element = %d\n", result);
    return 0;
}
```

Output=

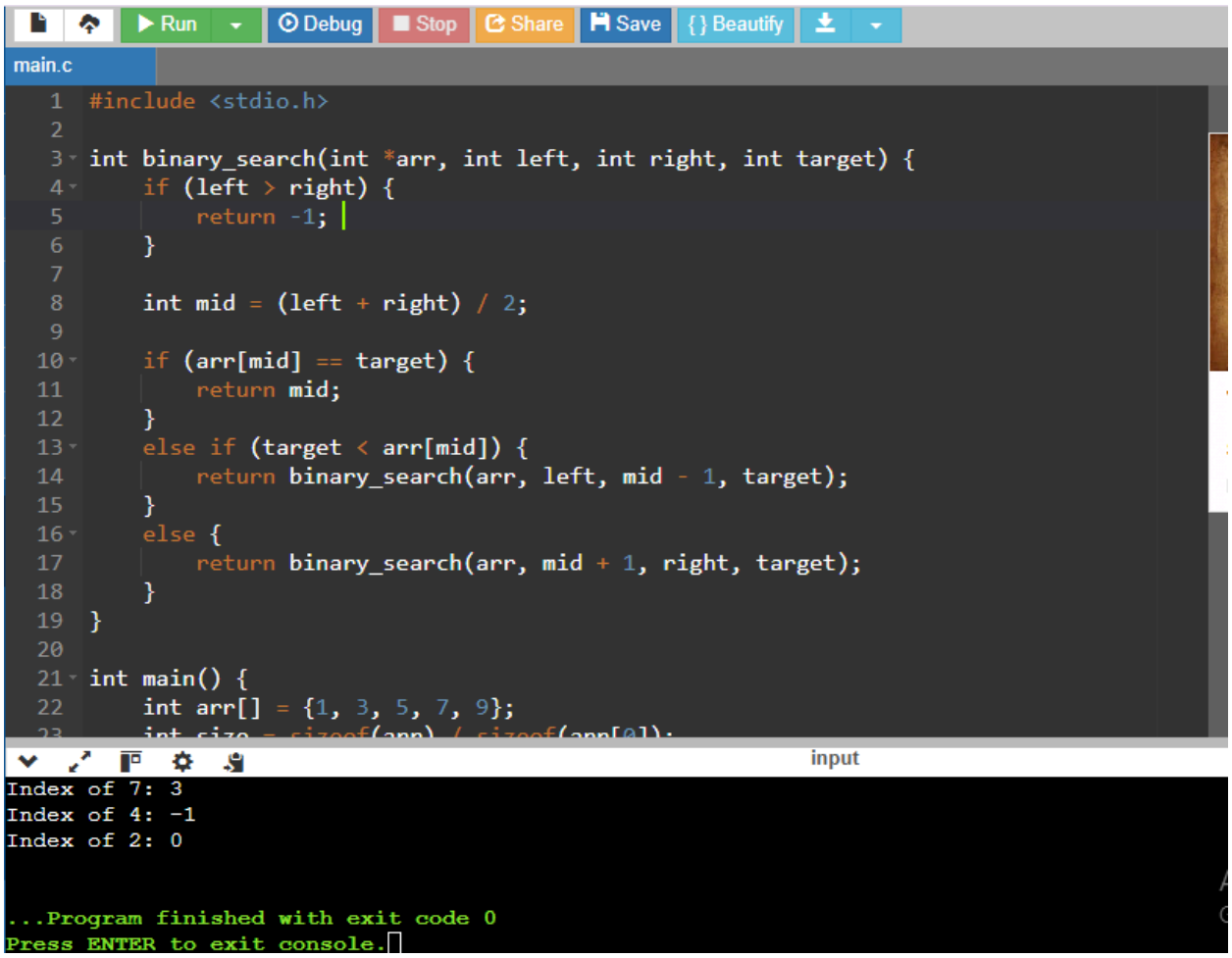


Problem 4.8

```
#include <stdio.h>
  int binary_search(int *arr, int left, int right, int target) {
    if (left > right) {
        return -1;
    }
    int mid = (left + right) / 2;
    if (arr[mid] == target) {
        return mid;
    }
    else if (target < arr[mid]) {
        return binary_search(arr, left, mid - 1, target);
    }
    else {
        return binary_search(arr, mid + 1, right, target);
    }
}
int main() {
    int arr[] = {1, 3, 5, 7, 9};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target1 = 7;
    int target2 = 4;
    int result1 = binary_search(arr, 0, size - 1, target1);
    int result2 = binary_search(arr, 0, size - 1, target2);
    printf("Index of %d: %d\n", target1, result1);
```

```
    printf("Index of %d: %d\n", target2, result2);
    int arr2[] = {2};
    int result3 = binary_search(arr2, 0, 0, 2);
    printf("Index of 2: %d\n", result3);
     return 0;
}
```

Output=



Problem 5

```
#include <stdio.h>
void merge(int *arr, int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int k = 0;
    int temp[right - left + 1];
    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    while (i <= mid) {
        temp[k++] = arr[i++];
    }
    while (j <= right) {
        temp[k++] = arr[j++];
    }
    for (i = left, k = 0; i <= right; i++, k++) {
        arr[i] = temp[k];
    }
}
void merge_sort(int *arr, int left, int right) {
```

```
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    merge_sort(arr, left, mid);
    merge_sort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int size = sizeof(arr) / sizeof(arr[0]);

    merge_sort(arr, 0, size - 1);
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
   return 0;
}
```

Output

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>

void merge(int *arr, int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int k = 0;

    int temp[right - left + 1];

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
```

input

```
3 9 10 27 38 43 82

...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 6

```c
#include <stdio.h>

  int swap_count = 0;

  void swap(int *a, int *b) {

   int temp = *a;

   *a = *b;

   *b = temp;

   swap_count++;
```

```
}
int partition(int *arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
     for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quick_sort(int *arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}
void print_array(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
```

```
    printf("\n");
}
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    print_array(arr, n);
    quick_sort(arr, 0, n - 1);
    printf("\nSorted array:\n");
    print_array(arr, n);
    printf("\nTotal swaps: %d\n", swap_count);
    return 0;
}
```

Output

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
int swap_count = 0;
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
    swap_count++;
}
int partition(int *arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quick_sort(int *arr, int low, int high) {
    if (low < high) {
```

input

```
1 5 7 8 9 10

Total swaps: 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 7

#include <stdio.h>

```
typedef struct {
    int weight;
    int value;
} Item;
void swap(Item *a, Item *b) {
    Item temp = *a;
    *a = *b;
    *b = temp;
}
void sort_by_ratio(Item *items, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            double r1 = (double)items[j].value / items[j].weight;
            double r2 = (double)items[j + 1].value / items[j + 1].weight;

            if (r1 < r2) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }
}
double fractional_knapsack(Item *items, int n, int capacity) {
    sort_by_ratio(items, n);
    double total_value = 0.0;
    int current_weight = 0;
```

```
    for (int i = 0; i < n; i++) {
      if (current_weight + items[i].weight <= capacity) {
          current_weight += items[i].weight;
          total_value += items[i].value;
      }
      else {

          int remain = capacity - current_weight;
          total_value += items[i].value * ((double)remain / items[i].weight);
          break;
      }
    }
    return total_value;
}
int main() {
    Item items[] = {
        {10, 60},
        {20, 100},
        {30, 120}
    };
    int n = 3;
    int W = 50;
    double result = fractional_knapsack(items, n, W);
    printf("%.2f\n", result);
    return 0;
```

}

Output

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
typedef struct {
    int weight;
    int value;
} Item;
void swap(Item *a, Item *b) {
    Item temp = *a;
    *a = *b;
    *b = temp;
}
void sort_by_ratio(Item *items, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            double r1 = (double)items[j].value / items[j].weight;
            double r2 = (double)items[j + 1].value / items[j + 1].weight;

            if (r1 < r2) {
                swap(&items[j], &items[j + 1]);
            }
        }
    }
}
double fractional_knapsack(Item *items, int n, int capacity) {
```

input

```
240.00


...Program finished with exit code 0
Press ENTER to exit console.
```

Problem 8a

```
#include <stdio.h>
#include <limits.h>
#define MAX 100
int minKey(int key[], int mstSet[], int n) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < n; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }
    return min_index;
}
void printMST(int parent[], int graph[MAX][MAX], int n) {
    int totalWeight = 0;
    printf("Edge \tWeight\n");
    for (int i = 1; i < n; i++) {
        printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);
        totalWeight += graph[i][parent[i]];
    }
   printf("Total MST Weight = %d\n", totalWeight);
}
void prims_mst(int graph[MAX][MAX], int n) {
    int parent[MAX];
```

```
    int key[MAX];
    int mstSet[MAX];
    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, mstSet, n);
        mstSet[u] = 1;
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
    printMST(parent, graph, n);
}
```

Problem 9a

```
#include <stdio.h>
#include <limits.h>
#define V 5
int minDistance(int dist[], int visited[]) {
    int min = INT_MAX, min_index = -1;
     for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }
    return min_index;
}
void dijkstra(int graph[V][V], int src) {
    int dist[V];
    int visited[V];
    for (int i = 0; i < V; i++) {
        dist[i] = INT_MAX;
        visited[i] = 0;
    }
    dist[src] = 0;
    for (int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited);
        visited[u] = 1;
```

```
        for (int v = 0; v < V; v++) {
            if (!visited[v] &&
                graph[u][v] &&
                dist[u] != INT_MAX &&
                dist[u] + graph[u][v] < dist[v]) {
               dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}
int main() {
    int graph[V][V] = {
        {0, 4, 0, 0, 8},
        {4, 0, 8, 0, 11},
        {0, 8, 0, 7, 0},
        {0, 0, 7, 0, 9},
        {8, 11, 0, 9, 0}
    };
    dijkstra(graph, 0);
```

```
    return 0;
}
```

Output

Problem 9b

```
#include <stdio.h>
#include <limits.h>
#define INF (INT_MAX / 2)
typedef struct {
    int u, v, w;
} Edge;
void bellman_ford(int V, int E, Edge *edges, int src) {
    int dist[V];
    for (int i = 0; i < V; i++)
        dist[i] = INF;
    dist[src] = 0;
    for (int i = 1; i <= V - 1; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    for (int j = 0; j < E; j++) {
        int u = edges[j].u;
        int v = edges[j].v;
```

```
        int w = edges[j].w;
        if (dist[u] != INF && dist[u] + w < dist[v]) {
            printf("Graph contains a negative-weight cycle\n");
            return;
        }
    }
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < V; i++) {
        if (dist[i] == INF)
            printf("%d\tINF\n", i);
        else
            printf("%d\t%d\n", i, dist[i]);
    }
}
int main() {
    int V = 5;
    int E = 10;
    Edge edges[] = {
        {0,1,6}, {0,2,7}, {1,2,8}, {1,3,5}, {1,4,-4},
        {2,3,-3}, {2,4,9}, {3,1,-2}, {4,0,2}, {4,3,7}
    };
    bellman_ford(V, E, edges, 0);
    return 0;
}
```

Output

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
#include <limits.h>
#define INF (INT_MAX / 2)
typedef struct {
    int u, v, w;
} Edge;
void bellman_ford(int V, int E, Edge *edges, int src) {
    int dist[V];
    for (int i = 0; i < V; i++)
        dist[i] = INF;
    dist[src] = 0;
    for (int i = 1; i <= V - 1; i++) {
        for (int j = 0; j < E; j++) {
            int u = edges[j].u;
            int v = edges[j].v;
            int w = edges[j].w;
            if (dist[u] != INF && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }
    for (int j = 0; j < E; j++) {
        int u = edges[j].u;
```

input

```
Vertex  Distance from Source
0       0
1       2
2       7
3       4
4       -2
```

Problem 10a

```
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int knapsack_01(int *weights, int *values, int n, int W) {
    int dp[n + 1][W + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {

            if (weights[i - 1] <= w) {
                dp[i][w] = max(
                    dp[i - 1][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
```

```
        }
    }

    return dp[n][W];
}
int main() {
    int weights[] = {2, 3, 4, 5};
    int values[]  = {3, 4, 5, 6};
    int n = 4, W = 5;
    printf("%d\n", knapsack_01(weights, values, n, W));
    return 0;
}
```

Output

Run Debug Stop Share Save { } Beautify

main.c

```c
#include <stdio.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
int knapsack_01(int *weights, int *values, int n, int W) {
    int dp[n + 1][W + 1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
        }
    }
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {

            if (weights[i - 1] <= w) {
                dp[i][w] = max(
                    dp[i - 1][w],
                    dp[i - 1][w - weights[i - 1]] + values[i - 1]
                );
            } else {
                dp[i][w] = dp[i - 1][w];
            }
```

input

```
7


...Program finished with exit code 0
Press ENTER to exit console.
```